Griffin Kuchar - Thrust Neural Network - Project Discussion

After implementing my Softmax Classifier sequentially for CPU computing (./cpumain) and in parallel for GPU computing (./main), I executed my programs with this Slurm batch script:

```
#!/bin/bash
#SBATCH --account=bfzu-delta-gpu
#SBATCH --job-name=thrustnn
#SBATCH --output=thrust_output_%j.txt
#SBATCH --error=thrust_error_%j.txt
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=gpu:1
#SBATCH --time=02:30:00
#SBATCH --mem=4G

# Compile
make clean
make all

# Run
sleep 2

# Small dataset
./bin/main data/blobs2d_3class.csv 200
./bin/cpumain data/blobs2d_3class.csv 200

# Large dataset
./bin/main data/large_blobs2d_3class.csv 50
./bin/cpumain data/large_blobs2d_3class.csv 50
```

Both programs first executed a softmax classifier with 200 training epochs on *blobs2d_3class.csv*, a 150-entry data set that included 2 float parameters and an integer target class value of 0, 1, or 2. Both programs then executed a softmax classifier with 50 epochs on a similar data set, large*_blobs2d_3class.csv* (same input parameters and target classes), with 3000 entries. Other hyperparameters included a learning rate of 0.1 and a batch size of 32. The output log from the run is as follows:

```
Softmax Accuracy: 99.33%
Dataset: data/blobs2d_3class.csv
Softmax Classifier GPU time: 12.97 seconds
1 Hidden Layer MLP Accuracy: 100.00%
MLP GPU time: 57.38 seconds
MLP was 0.67% percent more accurate than Softmax

CPU Softmax Accuracy: 99.33%
Dataset: data/blobs2d_3class.csv
CPU Softmax time: 2.47 ms
```

```
Softmax Accuracy: 95.90%
Dataset: data/large_blobs2d_3class.csv
Softmax Classifier GPU time: 75.67 seconds
1 Hidden Layer MLP Accuracy: 95.93%
MLP GPU time: 333.38 seconds
MLP was 0.03% percent more accurate than Softmax

CPU Softmax Accuracy: 95.90%
Dataset: data/large_blobs2d_3class.csv
CPU Softmax time: 14.02 ms
```

Observing the log, we notice that for the 150-entry dataset, the CPU Softmax Classifier was 5,251 times faster than its GPU counterpart, and the 3,000-entry dataset followed suit, with the CPU speedup over the GPU at 5,397. This is abnormal, and I can identify three GPU bottlenecks that explain this phenomenon:

1. **Low GPU Function Utilization:** In my parallelized program, the only segments that fully utilize a GPU's throughput are small, device-to-device data copies, and mathematical vector functions (such as *dot()* or *softmax()*). However, the vector functions operate on vectors of size 2, gaining a minimal performance increase over their sequential function counterparts.
2. **Excessive Implicit Data Transfers:** Throughout my GPU program, multiple repeated scalar indexes into device vectors are made (ie, *d_probs[c]* and *d_b[c]*). Naively, I did not realize accesses of such trigger implicit device-to-host data transfers. Any host-to-device or device-to-host transfers are bounded by being sequential in nature, and the vast amount of these accesses I implemented into my program quickly accumulated program execution time without any added benefit.
3. **"Ghost" Batching:** As previously noted, I implemented a batched epoch solution into my training loop. However, after further analysis, the batching added no GPU speedup.

Instead of properly batching and running each batch in parallel, I simply chunk the epoch into batches and sequentially train on each entry in the batch.

Possible further optimizations mapping to each bottleneck:

1. **Run Softmax Model on Data with Large Quantities of Input Parameters**

   → Low GPU Function Utilization

   In the current model, the mathematical functions are parallelly optimized, meaning a function on a vector of 2 will run in the same time on a vector of 1,000,000. A simple, yet significant GPU speedup can be achieved by running the current Softmax model on a dataset with many more input parameters. The kernel launch cost is equal regardless of vector size, so there will be no drawback in this aspect either.

2. **Restructure Scalar Accesses to Use Bulk Host Copies**

   → Excessive Implicit Data Transfers

   A simple optimization is to copy entire vectors to the host once per sample using thrust::copy, then perform all scalar arithmetic on the host side, reducing hundreds of individual transfers to a single bulk transfer per sample.

3. **Parallelize All Dot Operations within A Batch**

   → "Ghost" Batching

   By computing all dot products in the batch simultaneously using a matrix multiply, the model would accumulate all gradients across the whole batch, then do one weight update.